

643. Maximum Average Subarray I

Easy

Topics

Companies

You are given an integer array `nums` consisting of `n` elements, and an integer `k`.

Find a contiguous subarray whose length is equal to `k` that has the maximum average value and return *this value*. The answer will be accepted with a calculation error less than 10^{-5} .

Example 1:

Input: `nums = [1,12,-5,-6,50,3]`, `k = 4`

Output: `12.75000`

Explanation: Maximum average is $(12 + 50) / 4 = 51 / 4 = 12.75$

Example 2:

Input: `nums = [5]`, `k = 1`

Output: `5.00000`

Constraints:

- $n == \text{nums.length}$
- $1 \leq k \leq n \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

UNLOCK EXPERIENCES

Prepare your ne
today and tomo

Seen this question in a real interview before? 1/6

4.5K



225



Accepted

128 / 128
testcases
passed

Analysis

So



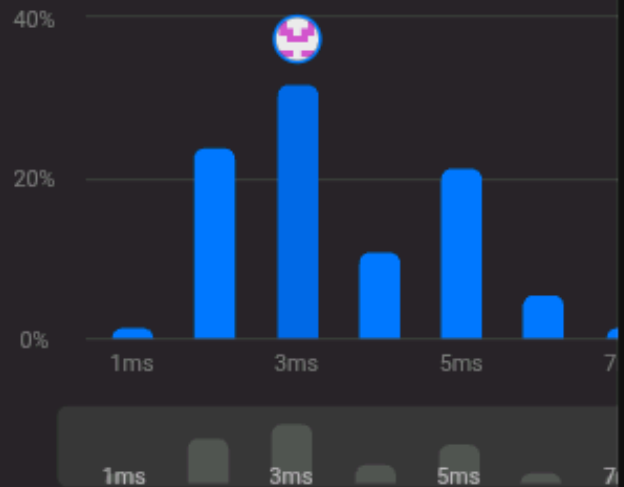
submitted at Jul 02, 2

Runtime

3 ms | Beats 75.69%

Memory

69.20 MB | Beats 94.84%



Code | Java

```
1 class Solution {
2     public double findMaxAverage(
3         int sum = 0;
4         for(int i = 0; i < k; i ++
5             sum += nums[i];
6         int maxSum = sum;
7         for(int i = k; i < nums.le
8             sum += nums[i] - nums[
```

View more

Write your notes here

Submissions | Description

1343. Number of Subarrays of Size K and Average Greater than Equal to Threshold

Medium Topics Companies

Given an array of integers `arr` and two integers `k` and `threshold`, return the number of subarrays of size `k` whose average is greater than or equal to `threshold`.

Example 1:

Input: `arr = [2,2,2,2,5,5]`, `k = 3`, `threshold = 4`

Output: 3

Explanation: Sub-arrays `[2,2,2]`, `[2,2,5]` and `[2,5,5]` have averages `2`, `3` and `4` respectively. All other sub-arrays of size `3` have averages less than `4` (the threshold).

Example 2:

Input: `arr = [11,13,17,23]`, `k = 3`, `threshold = 5`

Output: 6

Explanation: The first 6 sub-arrays of size `3` have averages greater than `5` (the threshold). The last sub-array has an average of `5` (the threshold) and is not counted.

Constraints:

- $1 \leq arr.length \leq 10^3$
- $1 \leq arr[i] \leq 10^4$

Editorial | Accepted x

All Submissions

Accepted 69 / 69 testcases passed

Mohanapriya2212 submitted at Jul 02, 2026 19:45

Analysis

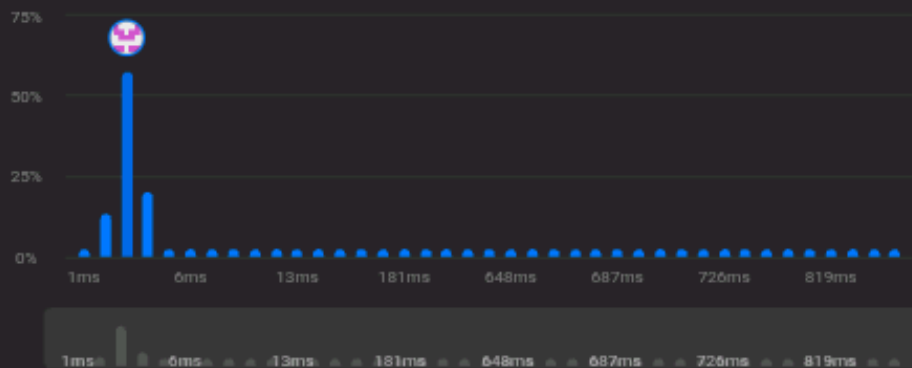
Solution

Runtime

3 ms | Beats 86.00%

Memory

70.25 MB | Beats 98.80%



Code | Java

```
1 class Solution {
2     public int numOfSubarrays(int[] a, int k, int t) {
3         int cnt=0;
4         int sum=0;
5         for(int i=0;i<k;i++){
6             sum =sum+a[i];
7         }
8         int avg=sum/k;
```

View more

Write your notes here

Code

35. Search Insert Position

Solved 

Easy

 Topics Companies

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [1,3,5,6], target = 5`
Output: 2

Example 2:

Input: `nums = [1,3,5,6], target = 2`
Output: 1

Example 3:

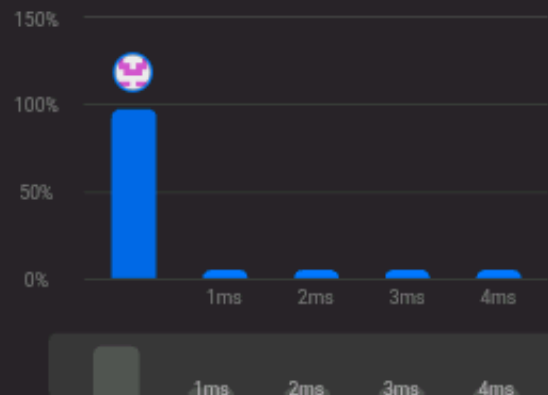
Input: `nums = [1,3,5,6], target = 7`
Output: 4

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` contains **distinct** values sorted in **ascending** order.
- $-10^4 \leq \text{target} \leq 10^4$

All Submissions 

Accepted

66 / 66
testcases
passed Analysis Solution submitted at Jul 02, 2 Runtime0 ms | Beats 100.00%  Memory44.44 MB | Beats 92.12% 

Code | Java

```
1 class Solution {
2     public int searchInsert(int[] A, i
3         int n=A.length;
4         int low=0;
5         int high=n-1;
6         while(low<=high){
7             int mid=low+(high-low)/2;
8             if(t==A[mid]) return mid;
```

Search in Rotated Sorted Array

Medium

Topics

Companies

There is an integer array `nums` sorted in ascending order (with distinct values).

Prior to being passed to your function, `nums` is possibly left rotated at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`
Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`
Output: -1

Example 3:

Input: `nums = [1]`, `target = 0`
Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- All values of `nums` are unique.
- `nums` is an ascending array that is possibly rotated.

30.5K 689 223 Online

Accepted

196 / 196
testcases
passed

Analysis

Solution

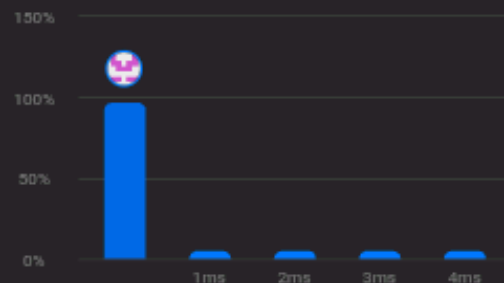
submitted at Jul 02, 2

Runtime

0 ms | Beats 100.00%

Memory

43.62 MB | Beats 81.85%



Code | Java

```
1 class Solution {
2     static int search(int[] arr , int t
3         int low = 0;
4         int high = arr.length-1;
5         while (low <= high) {
6             int mid = low + (high - lo
7             if(arr[mid] == target){
8                 return mid;
```

View more

Write your notes here

26. Remove Duplicates from Sorted Array

Easy

Topics

Companies

Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order of the elements** should be kept the **same**.

Consider the number of *unique elements* in `nums`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements of `nums` from index `k` to the end of the array can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length
```

```
int k = removeDuplicates(nums); // Calls your implementation
```

```
assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1:

Input: `nums = [1,1,2]`

← All Submissions

Accepted 362 / 362 testcases passed

Mohanapriya2... submitted at Jul 06, 2026 07:43

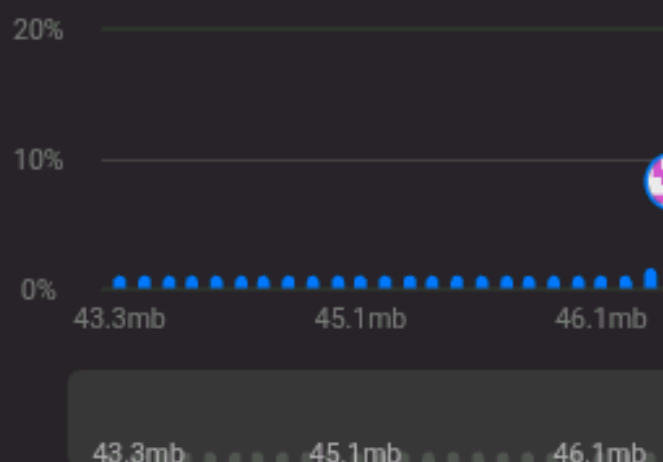
Runtime

1 ms | Beats 78.53%

Memory

46.48 MB | Beats 91.71%

15.06% of solutions used 46.9 MB of memory



Code | Java

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int j = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[j-1]) {
                nums[j] = nums[i];
                j++;
            }
        }
        return j;
    }
}
```

View more

[Time taken: 1d 19hrs 27m 39s]